Example implementation

To migrate from the legacy Fitbit Web API to Google Health API, you will move from general OAuth2 libraries to the Google Auth Library. Following is an architectural suggestion and a pseudo-code implementation written in Javascript to handle this "Dual-Library" state.

1. The "Middleware Switch"

Since you cannot migrate all users at once, your backend needs to determine which library to use based on the user's current **`apiVersion`** stored in your database.

Implementation

```
const { OAuth2Client } = require('google-auth-library');
const FitbitV1Strategy = require('fitbit-oauth2-library').Strategy;

// 1. Initialize the Google Health API Client
const GHAClient = new OAuth2Client(
  process.env.GOOGLE_CLIENT_ID,
  process.env.GOOGLE_CLIENT_SECRET,
  process.env.REDIRECT_URI
);

// 2. Create a Unified Fetcher
async function fetchSteps(user) {
  if (user.apiVersion === 4) {
    // ---- GOOGLE OAUTH LIBRARY LOGIC ----
    GHAClient.setCredentials({ refresh_token: user.refreshToken });
    const url = 'GET https://health.googleapis.com/v4/users/me/dataTypes/
    const res = await GHAClient.request({ url });
    return res.data;
  } else {
    // ---- FITBIT WEB API LEGACY LOGIC ----
    // Use your existing Fitbit open-source library logic here
    return callLegacyV1Api(user.accessToken);
  }
}
```

2. Migrate the UX flow

To maximize retention, use an "**Interrupt-and-Upgrade**" pattern. This ensures the user isn't forced to re-login until they are already engaged with the app.

Redirect logic

When a Fitbit Web API user hits a specific feature, trigger the migration:

```
app.get('/dashboard', async (req, res) => {
  const user = await db.users.find(req.user.id);

  if (user.apiVersion === 1) {
    // Render a "soft" migration page explaining the Google transition
    return res.render('migrate-to-google', {
      title: "Keep your data syncing",
      message: "Fitbit is moving to Google accounts. Re-connect now to st
    });
  }

  const data = await fetchSteps(user);
  res.render('dashboard', { data });
});
```

3. Key technical transitions

When writing your JavaScript migration scripts, keep these differences in mind:

Feature	Fitbit Web API (Legacy)	Google Health API (Google-Identity)
Token Endpoint	https://api.fitbit.com/oauth2/token	https://oauth2.googleapis.com/token
Auth Library	Standard Open Source	Google Auth
Scope	activity	https://www.googleapis.com/auth/googlehealth.activi

User ID	Fitbit Encoded ID returned in /oauth2/token response	User ID returned from users.getIdentity endpoint

4. Retention checklist

- **Session Persistence**: Do not clear the user's old Fitbit Web API session until the Google Health API access_token is successfully verified and saved to your database.
- **Auto-Revoke**: Once the Google Health API migration is complete, use a POST request to the legacy Fitbit revoke endpoint: https://api.fitbit.com/oauth2/revoke. This ensures the user doesn't see "duplicate" app permissions in their Fitbit settings.
- **Error Handling**: If a Fitbit call returns a 401 Unauthorized, automatically redirect to the Google OAuth flow instead of showing an error message.

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-06-06 UTC.